Как устроен код проекта: простое объяснение

Здесь описано, что лежит в каждом файле и зачем он нужен. Разбирать каждую строчку не нужно, достаточно понимать общую идею.

Главная идея в одной картинке

Проект работает как конвейер из четырёх шагов:

```
Шаг 1              Шаг 2              Шаг 3                    Шаг 4
СОБРАТЬ            НАРЕЗАТЬ           ОТВЕЧАТЬ                 ПРОВЕРИТЬ
статьи из      →   статьи на      →   на вопросы:          →   насколько хорошо
Википедии          карточки           библиотекарь +           всё работает
                                      рассказчик               (эксперименты)
```

- **Библиотекарь** — часть программы, которая ищет подходящие карточки.
- **Рассказчик** — нейросеть, которая пишет ответ по этим карточкам.

Шаги 1 и 2 нужно сделать один раз. Результат уже лежит в папке `data`.

Карта папок

Папка или файл	Что там лежит
`data/raw/`	20 скачанных статей в виде обычных текстовых файлов
`data/chunks.json`	все 303 карточки (кусочки текста)
`data/sources.json`	список скачанных статей и их адреса в интернете
`research/questions.json`	50 вопросов с правильными ответами
`research/results/`	таблицы с результатами экспериментов
`research/charts/`	графики для работы и презентации
`docs/`	готовые PDF-файлы и видео о проекте
`tutorial/`	программы, которые сделали обучающее видео (на защите не нужны)

Файлы с кодом по порядку

`config.py` — настройки

Это «пульт управления» проекта. Здесь записаны:

- **список статей**, которые нужно скачать (`SOURCES`): на каком сайте и как называется статья;
- **длина карточки** (`CHUNK_SIZE = 700`): максимум символов в одной карточке;
- **сколько карточек давать нейросети** (`TOP_K = 3`);
- **какую нейросеть использовать** (`MODEL = "qwen2.5:1.5b"`);
- адрес программы Ollama (`OLLAMA_URL`) и пути к папкам.

Если нужно что-то поменять, например взять другую нейросеть, меняют только этот файл.

`step1_collect.py` — Шаг 1: сбор статей (парсер)

Что делает: скачивает 20 статей из Википедии и Викицитатника и сохраняет из них чистый текст.

Как работает:

1. `download_html` — скачивает страницу статьи. Если сайт отвечает «слишком много запросов» (код 429), программа ждёт и пробует снова.
2. `parse_article` и `html_to_lines` — разбирают страницу. Страница сайта написана на языке HTML, где кроме текста много служебного. Программа с помощью библиотеки **BeautifulSoup** достаёт заголовки, абзацы, списки и таблицы.
3. Служебные разделы («Примечания», «Ссылки», «Литература») выбрасываются, они перечислены в `SKIP_SECTIONS`.
4. `main` — проходит по всем статьям, сохраняет каждую в папку `data/raw/` и делает паузу 3 секунды, чтобы не нагружать сайт.

Результат: 20 текстовых файлов в `data/raw/`.

`step2_split.py` — Шаг 2: нарезка на карточки

Что делает: режет каждую статью на небольшие кусочки — карточки.

Зачем: нейросеть не может прочитать сразу всю библиотеку (150 тысяч символов). Поэтому ей дают только несколько подходящих карточек.

Как работает:

1. Статья делится на разделы по заголовкам.
2. `make_chunks` складывает абзацы в карточку, пока она не станет длиннее 700 символов, и начинает новую. Слишком длинный абзац режется по предложениям.
3. Каждая карточка запоминает: **из какой статьи** (`source`), **из какого раздела** (`section`), **адрес** (`url`) и **сам текст** (`text`).

Результат: файл `data/chunks.json` с 303 карточками.

`rag.py` — сердце проекта (библиотекарь + рассказчик)

Это самый важный файл. В нём две части.

Часть 1. Библиотекарь — класс `Library`

Он ищет карточки, которые лучше всего подходят к вопросу.

Сначала текст готовится к поиску:

- `words_of` — разбивает текст на слова, делает все буквы маленькими, заменяет «ё» на «е» и выбрасывает **стоп-слова** — частые слова без смысла вроде «в», «и», «какой», «году» (список `STOP_WORDS`).

- `stems_of` — обрезает слова до **основы**: «журавлях» → «журавл». Для этого используется библиотека **snowballstemmer**.

Затем библиотекарь считает для каждой карточки **оценку** — насколько она подходит. Есть три способа (функция `_score`):

Способ	Как считает
`words` — совпадение слов	сколько слов вопроса есть в карточке буква в букву
`stems` — совпадение основ	то же, но по основам слов, поэтому «Гамзатова» = «Гамзатову»
`bm25` — BM25	по основам, но редкие слова (например, «Хиросима») дают больше очков, чем частые (например, «поэт»), а длинная карточка не получает преимущества только за длину

Функция `search` берёт карточки с самой высокой оценкой (по умолчанию 3).

Часть 2. Рассказчик — функции `ask_model` и `answer`

- `ask_model` — отправляет текст нейросети через программу **Ollama** и получает ответ. Параметр `temperature: 0` делает так, чтобы на один и тот же вопрос нейросеть всегда отвечала одинаково.
- `answer` — главная функция. Она умеет отвечать двумя способами:
- **без библиотеки** — просто задаёт вопрос нейросети. В эксперименте к вопросу добавляется просьба сказать «Я не знаю», если нейросеть не уверена (шаблон `PROMPT_NO_LIBRARY`). Если передать `allow_unknown=False`, отправляется только вопрос, как обычному чат-боту (шаблон `PROMPT_PLAIN`), и тогда нейросеть обычно выдумывает ответ;
- **с библиотекой (RAG)** — сначала находит 3 карточки, вставляет их в вопрос и просит нейросеть отвечать **только по ним** (шаблон `PROMPT_WITH_LIBRARY`). Если ответа нет, нейросеть должна сказать: «В моей библиотеке нет ответа».

Шаблоны — это заготовки текста для нейросети, куда программа подставляет вопрос и карточки.

`step3_ask.py` — помощник в консоли

Простая программа для проверки: в чёрном окне (консоли) вводишь вопрос и получаешь ответ и список источников. Пустая строка — выход. Внутри она просто вызывает `answer` из `rag.py`.

`app.py` — помощник в браузере (для показа на защите)

Тот же помощник, но в красивом окне браузера. Сделан с помощью библиотеки **Streamlit**, которая позволяет создать веб-страницу на Python за несколько строк.

Что есть на странице:

- поле для вопроса, как в чате;

- слева — выбор нейросети и переключатель **«Подглядывать в библиотеку (RAG)»**. Его можно выключить прямо на защите и показать разницу;
- ещё один переключатель — **«Разрешить ответ „Я не знаю“»**. Он работает, только когда библиотека выключена. По умолчанию он выключен: нейросеть получает просто вопрос и выдумывает ответ. Если его включить, нейросеть честно скажет «Я не знаю»;
- под ответом — **ссылки на источники**. Если библиотека выключена, появляется предупреждение, что ответ может быть выдумкой.

Запуск:

```
streamlit run app.py
```

`step4_experiments.py` — Шаг 4: эксперименты

Этот файл проверяет, насколько хорошо всё работает, и рисует графики.

Эксперимент «поиск» (`run_search_experiment`): для каждого из 45 обычных вопросов проверяет, нашёл ли каждый из трёх способов поиска карточку с ответом среди 1, 3 и 5 карточек. Результат сохраняется в `research/results/search.csv`.

Эксперименты «ответы» (`run_answer_experiments`): задаёт вопросы нейросети без библиотеки и с библиотекой и записывает ответы в `research/results/answers.csv`. Если программу прервать, при следующем запуске она продолжит с того же места.

Проверка ответов (`grade`):

- в ответе есть ключевое слово из правильного ответа → **«Верно»**;
- нейросеть написала «не знаю», «нет ответа» и т. п. (список `REFUSALS`) → **«Не знаю»**;
- иначе → **«Ошибка (выдумка)»**;
- для вопросов-ловушек верным считается ответ «нет» или «не знаю».

Отчёт (`make_report`): считает итоги в `summary.csv` и рисует три графика с помощью библиотеки **matplotlib**:

- `chart_search.png` — сравнение способов поиска;
- `chart_answers.png` — ответы без библиотеки и с ней;
- `chart_topics.png` — верные ответы по темам.

Режим `--quick` берёт только 15 вопросов и одну модель, чтобы эксперимент шёл около 10 минут, а не 1–2 часа. Именно в этом режиме получены результаты в работе.

`make_docs.py` — создание PDF

Собирает PDF-файлы исследовательской работы и руководства (папка `docs/`) с помощью библиотеки **reportlab**. Для защиты разбираться в нём не нужно.

`requirements.txt` — список библиотек

Список готовых библиотек, которые нужны проекту. Устанавливаются одной командой:

```
pip install -r requirements.txt
```

Библиотека	Для чего
`requests`	отправлять запросы в интернет и в Ollama
`beautifulsoup4`	разбирать HTML-страницы (парсер)
`snowballstemmer`	обрезать слова до основы
`matplotlib`	рисовать графики
`streamlit`	делать страницу помощника в браузере
`reportlab`	создавать PDF

Словарик

Слово	Простыми словами
RAG	экзамен с открытой книгой: сначала найти текст, потом по нему ответить
Нейросеть (языковая модель)	программа, которая обучилась на огромном количестве текстов и умеет их продолжать
Галлюцинация	правдоподобный, но выдуманный ответ нейросети
Ollama	бесплатная программа для запуска нейросетей на своём компьютере
Парсер	программа, которая достаёт нужный текст из страницы сайта
HTML	язык, на котором написаны страницы сайтов
Карточка (чанк)	небольшой кусочек текста из статьи
Стоп-слова	частые слова без смысла: «в», «и», «какой»
Стемминг (основа слова)	обрезание окончаний: «журавлях» → «журавл»
BM25	способ поиска, в котором редкие слова важнее частых
Промпт (шаблон)	текст-задание, который мы отправляем нейросети
temperature	«случайность» нейросети; 0 — всегда одинаковый ответ
JSON, CSV	форматы файлов для хранения данных: JSON — списки и записи, CSV — таблицы
Функция	именованный кусок кода, который делает одно действие (`def ...`)
Класс	«чертёж» объекта, который хранит данные и умеет что-то делать (`class Library`)